

Requirements

Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota.

Il DDR si deve fermare quando tornato in HOME.

Il DDR è anche sensibile ai dati rilevati da un Radar.

Quando il Radar rileva un 'intruso' a una distanza minore di una distanza prefissata DMIN:

- se il DDR si sta muovendo, il DDR si ferma fino a che il Radar non rileva più un 'intruso' così vicino;
- se il DDR è fermo in HOME, il DDR ruota su sè stesso fino a che un 'intruso' così vicino scompare.

Requirement analysis

Che cosa intende il committente per Differential Drive Robot (DDR)?

Un DDR è un tipo di robot mobile il cui movimento si basa su due ruote azionate separatamente posizionate su entrambi i lati del corpo del robot. Tramite tali ruote quindi esso può cambiare direzione variando la velocità relativa di rotazione delle sue ruote, non richiedendo un ulteriore movimento di sterzata.

I requisiti dicono di modellare un sistema che controlli il robot, questo implica che il robot non è un'entità proattiva, bensì reattiva agli stimoli indotti dal controllore. Pertanto il robot non è altro che un servizio messo a disposizione del controllore. Per modellare il comportamento del robot utilizziamo il linguaggio di modellazione QAK che ci permette di modellare in maniera formale entità come attori, riducendo l'abstraction gap della mera implementazione in un qualsiasi linguaggio di programmazione.

Parlando con il committente è emerso che il robot è inscrivibile in un quadrato e non è da creare da zero, è fornito dal committente stesso, modelliamo quindi tale entità nel nostro sistema seguendo quelle che sono le specifiche del robot del committente.

Analizzando tale robot è emerso che esso offre queste funzionalità:

1. move (per permettergli di muoversi in avanti);
2. turnRight (per permettergli di girarsi verso destra);
3. turnLeft (per permettergli di girarsi verso sinistra).

Come per le funzioni sono stati analizzati anche i segnali a cui esso risponde ed è emerso che essi sono formati in questo modo:

1. {"robotmove":"moveForward","time": TIME};
2. {"robotmove":"moveBackward","time": TIME};
3. {"robotmove":"turnRight","time": TIME};
4. {"robotmove":"turnLeft","time": TIME};
5. {"robotmove":"alarm","time":"1"}.

Da questa analisi approfondita è scaturito questo modello che rappresenta il robot nelle sue funzioni:

```

Event turnLeft : turnLeft(Time)
Event turnRight : turnRight(Time)
Event moveForward : moveForward(Time)
Event moveBackward : moveBackward(Time)
Event alarm : alarm(time)

QActor robot context ctxddr {
    [#
        var BodyDiameter = 2.0

        var CurrX = 0.0 // posizione in X
        var CurrY = 0.0 // posizione in Y
        var CurrRotationDegree = 0.0 // da 0 a 359.0

        var CurrTimer = 0L;

    #]
    State s0 initial {
        println("[DDR] ready!") color cyan
    }
    Goto waitCommand

    State waitCommand {
        println("[DDR] waiting for commands!") color magenta
    }
    Transition t0
        whenEvent turnLeft -> rotateLeft
        whenEvent turnRight -> rotateRight
        whenEvent moveForward -> moveForward
        whenEvent moveBackward -> moveBackward

    State rotateLeft {
        onMsg(turnLeft : turnLeft(Time)) {
            [# CurrTimer = payloadArg(0).toString().toDouble() #]
            println("[DDR] rotating Left in $CurrTimer ms") color cyan
        }
        [# CurrRotationDegree = (CurrRotationDegree - (90 / CurrTimer)) % 360 #]
    }
    Transition t0
        whenTimeVar CurrTimer -> waitCommand
        whenEvent alarm -> waitCommand

    State rotateRight {
        onMsg(turnRight : turnRight(Time)) {
            [# CurrTimer = payloadArg(0).toString().toDouble() #]
            println("[DDR] rotating Right in $CurrTimer ms") color cyan
        }
        [# CurrRotationDegree = (CurrRotationDegree + (90 / CurrTimer)) % 360 #]
    }
    Transition t0
        whenTimeVar CurrTimer -> waitCommand
        whenEvent alarm -> waitCommand

    State moveForward {
        onMsg(moveForward : moveForward(Time)) {
            [#
                CurrTimer = payloadArg(0).toString().toDouble()
                val radians = Math.toRadians(CurrRotationDegree)
                CurrX += ReceivedDistance * kotlin.math.cos(radians)
                CurrY += ReceivedDistance * kotlin.math.sin(radians)
            #]
            println("[DDR] moving Forward in $CurrTimer ms") color cyan
        }
    }
    Transition t0
        whenTimeVar CurrTimer -> waitCommand
        whenEvent alarm -> waitCommand

    State moveBackward {
        onMsg(moveBackward : moveBackward(Time)) {
            [#
                CurrTimer = payloadArg(0).toString().toDouble()
                val radians = Math.toRadians(CurrRotationDegree)
                CurrX -= ReceivedDistance * kotlin.math.cos(radians)

```

```

        CurrY -= ReceivedDistance * kotlin.math.sin(radians)
        #]
        println("[DDR] moving Backward in $CurrTimer ms") color cyan
    }
}
Transition t0
    whenTimeVar CurrTimer -> waitCommand
    whenEvent alarm -> waitCommand
}

```

Che cosa intende il committente per Stanza?

La stanza è lo spazio in cui il robot può muoversi, essa da specifica è definita come rettangolare e, quindi dotata di 4 lati in cui ogni coppia di lati paralleli ha stessa lunghezza. Pertanto per modellare la stanza useremo una coppia di variabili reali che indichino la lunghezza della base e dell'altezza della stanza. L'unità di misura utilizzata per misurare tali lati sarà l'unità robotica (lunghezza del lato del robot):

```

var StanzaB = 10.0
var StanzaH = 15.0

```

Facendo questo è come se (concettualmente) stessimo dividendo il piano della stanza in una griglia composta da celle di grandezza pari al quadrato che iscrive il robot.

Che cosa intende il committente per HOME?

La HOME è una posizione specifica all'interno della stanza da cui parte il robot all'inizio dell'esecuzione.

Per rappresentare tale entità utilizziamo un set di coordinate per indicare la posizione della HOME all'interno della stanza e un'ulteriore variabile per indicarne il diametro (pari a quello del robot stesso).

```

var HomeX = 0.0
var HomeY = 0.0
var HomeD = 2.0 // diametro del robot

```

Che cosa intende il committente per radar?

Il radar è un dispositivo che misura la distanza dall'oggetto davanti ad esso con una propria frequenza di campionamento, notificandola ai dispositivi che da esso dipendono.

Date queste osservazioni emerge quindi il seguente modello:

```
Event currMeasure : currMeasure(CurrDistance)
```

```
QActor sonar context ctxddr {  
  [#  
    var Timer = 0L // in ms  
    var CurrDistance = 0.0  
  #]  
  State s0 initial {  
    println("[SONAR] ready!") color yellow  
  }  
  Goto measure  
  
  State measure {  
    [# CurrDistance = java.util.Random().nextDouble(0.0, 20.0) #]  
    println("[SONAR] Current measure: $CurrDistance") color yellow  
    emit currMeasure : currMeasure($CurrDistance)  
  }  
  Transition t0  
    whenTimeVar Timer -> measure  
}
```

Problem analysis

Test plans

Project

Testing

Deployment

Maintenance

By Marcellini Alessandro | email: alessandr.marcellin2@studio.unibo.it |



GIT repo: <https://github.com/alessandromarcellini/iss2026>